

LangChain Handbook: Complete Developer Guide

Table of Contents

1. [Introduction](#)
2. [Installation & Setup](#)
3. [Core Components](#)
4. [Chains](#)
5. [Agents](#)
6. [Memory Management](#)
7. [RAG \(Retrieval Augmented Generation\)](#)
8. [Vector Stores & Embeddings](#)
9. [Tools & Integrations](#)
10. [Production Best Practices](#)

Introduction

LangChain is a framework for developing applications powered by large language models (LLMs). It simplifies every stage of the LLM application lifecycle through modular components and standardized interfaces.

Key Benefits

- **Composable by design:** Standard interface for experimenting with different providers
- **Real-time data augmentation:** Connect LLMs to live data sources and APIs
- **600+ integrations:** Extensive ecosystem of model providers, tools, and vector stores
- **Production-ready:** Used by companies like Rakuten, Cisco, and Moody's

Installation & Setup

Basic Installation

```
# Core LangChain
pip install langchain

# With specific integrations
pip install langchain-openai
```

```
pip install langchain-anthropic
pip install langchain-community
```

Environment Setup

```
import os
import getpass

# Set API keys
if not os.environ.get("OPENAI_API_KEY"):
    os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter OpenAI API key: ")

# Optional: LangSmith for tracing
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "your-langsmith-key"
```

Basic Usage Example

```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Initialize model
model = init_chat_model("gpt-4", model_provider="openai")

# Create a simple chain
prompt = ChatPromptTemplate.from_template("Tell me a joke about {topic}")
chain = prompt | model | StrOutputParser()

# Execute
result = chain.invoke({"topic": "AI"})
print(result)
```

Core Components

1. Schema

Defines fundamental data structures used across LangChain:

- **Text:** Primary communication medium
- **ChatMessages:** System, Human, and AI message types
- **Documents:** Unstructured data with content and metadata
- **Examples:** Input-output pairs for training/evaluation

2. Models

Interface for working with different LLMs:

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

# OpenAI
openai_model = ChatOpenAI(model="gpt-4", temperature=0.7)

# Anthropic
anthropic_model = ChatAnthropic(model="claude-3-sonnet")
```

3. Prompts

Structured templates for model inputs:

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["city"],
    template="Describe a perfect day in {city}?"
)

# Use with variables
formatted_prompt = prompt.format(city="Paris")
```

Chains

Chains link multiple LLMs or operations in sequence. The output of one step becomes input for the next.

Simple Chain Example

```
from langchain.chains import LLMChain
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a summary about {topic}"
)

chain = LLMChain(llm=model, prompt=prompt)
result = chain.run("Machine Learning")
```

Sequential Chains

```
from langchain.chains import SimpleSequentialChain

# Chain 1: Generate content
chain_one = LLMChain(
    llm=model,
    prompt=PromptTemplate(
        input_variables=["topic"],
        template="Generate key points about {topic}"
    )
)

# Chain 2: Summarize content
chain_two = LLMChain(
    llm=model,
    prompt=PromptTemplate(
        input_variables=["content"],
        template="Summarize these key points: {content}"
    )
)

# Combine chains
overall_chain = SimpleSequentialChain(
    chains=[chain_one, chain_two],
    verbose=True
)

result = overall_chain.run("Artificial Intelligence")
```

Modern Chain Syntax (LCEL)

```
# Using LangChain Expression Language
from langchain_core.output_parsers import StrOutputParser

chain = prompt | model | StrOutputParser()
result = chain.invoke({"topic": "Python automation"})
```

Agents

Agents use LLMs as reasoning engines to determine actions and execute them autonomously.

Agent Types

1. **OpenAI Functions:** Works with OpenAI function-calling models
2. **OpenAI Tools:** Designed for OpenAI built-in tools
3. **ReAct:** For simple models with reasoning and acting
4. **JSON Chat:** Optimized for JSON-based interactions

Creating an Agent

```
from langchain.agents import AgentExecutor, create_openai_functions_agent
from langchain.tools import Tool
from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

# Initialize tools
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

tools = [
    Tool(
        name="Wikipedia",
        description="Search Wikipedia for information",
        func=wikipedia.run
    )
]

# Create agent
from langchain import hub
prompt = hub.pull("hwchase17/openai-functions-agent")

agent = create_openai_functions_agent(model, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Execute
result = agent_executor.invoke({"input": "Who founded Tesla?"})
```

Memory Management

Memory allows applications to maintain context across interactions.

Memory Types

1. Conversation Buffer Memory

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
memory.save_context(
    {"input": "Hi, I'm Alice"},
    {"output": "Hello Alice, how can I help you?"}
)

print(memory.load_memory_variables({}))
```

2. Conversation Summary Memory

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(llm=model)
memory.save_context(
    {"input": "Tell me about machine learning"},
    {"output": "Machine learning is a subset of AI..."}
)
```

3. Entity Memory

```
from langchain.memory import ConversationEntityMemory

entity_memory = ConversationEntityMemory(llm=model)
# Automatically extracts and remembers entities from conversations
```

Memory in Chains

```
from langchain.chains import ConversationChain

conversation = ConversationChain(
    llm=model,
    memory=ConversationBufferMemory(),
    verbose=True
)

response1 = conversation.predict(input="Hi, I'm working on a Python project")
response2 = conversation.predict(input="Can you help me with error handling?")
```

RAG (Retrieval Augmented Generation)

RAG combines LLMs with external knowledge sources to provide accurate, contextual responses.

RAG Architecture

1. **Indexing:** Load, split, embed, and store documents
2. **Retrieval:** Find relevant documents for user queries
3. **Generation:** Use retrieved context to generate responses

Basic RAG Implementation

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA

# 1. Load and split documents
loader = TextLoader("your_document.txt")
documents = loader.load()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = text_splitter.split_documents(documents)

# 2. Create embeddings and vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# 3. Create retrieval chain
qa_chain = RetrievalQA.from_chain_type(
    llm=model,
    chain_type="stuff",
    retriever=vectorstore.as_retriever()
)

# 4. Query the system
response = qa_chain.run("What is the main topic of the document?")
```

Advanced RAG with Custom Retriever

```
from langchain.schema.retriever import BaseRetriever
from langchain_core.documents import Document

class CustomRetriever(BaseRetriever):
    def _get_relevant_documents(self, query: str) -> List[Document]:
        # Custom retrieval logic
        return [Document(page_content="Retrieved content")]

# Use custom retriever
custom_retriever = CustomRetriever()
qa_chain = RetrievalQA.from_chain_type(
    llm=model,
    retriever=custom_retriever
)
```

Vector Stores & Embeddings

Vector stores enable semantic search by storing and retrieving vector embeddings.

Embedding Models

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.embeddings import HuggingFaceEmbeddings

# OpenAI embeddings
openai_embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

# HuggingFace embeddings
hf_embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Vector Store Implementations

FAISS (Local)

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_texts(
    texts=["Document 1 content", "Document 2 content"],
    embedding=openai_embeddings
)

# Save and load
vectorstore.save_local("faiss_index")
new_vectorstore = FAISS.load_local("faiss_index", openai_embeddings)
```

Pinecone (Cloud)

```
from langchain_pinecone import PineconeVectorStore
import pinecone

# Initialize Pinecone
pinecone.init(api_key="your-api-key", environment="your-env")

vectorstore = PineconeVectorStore.from_texts(
    texts=["Document content"],
    embedding=openai_embeddings,
    index_name="your-index"
)
```


Chroma (Local/Cloud)

```
from langchain_chroma import Chroma

vectorstore = Chroma.from_texts(
    texts=["Document content"],
    embedding=openai_embeddings,
    persist_directory="./chroma_db"
)
```

Vector Search Operations

```
# Similarity search
results = vectorstore.similarity_search("query", k=3)

# Similarity search with scores
results_with_scores = vectorstore.similarity_search_with_score("query", k=3)

# Maximum marginal relevance search (diverse results)
mmr_results = vectorstore.max_marginal_relevance_search("query", k=3)
```

Tools & Integrations

LangChain provides 600+ integrations with external services and tools.

Built-in Tools

Search Tools

```
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

# DuckDuckGo search
search = DuckDuckGoSearchRun()

# Wikipedia
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
```

Web Scraping

```
from langchain_community.tools import WebReaderTool

web_reader = WebReaderTool()
content = web_reader.run("https://example.com")
```

Code Execution

```
from langchain_experimental.tools import PythonREPLTool

python_repl = PythonREPLTool()
result = python_repl.run("print('Hello, World!')")
```

Custom Tools

```
from langchain_core.tools import tool

@tool
def multiply_numbers(a: int, b: int) -> int:
    """Multiply two numbers together."""
    return a * b

@tool
def get_weather(city: str) -> str:
    """Get weather information for a city."""
    # Your weather API logic here
    return f"Weather in {city}: Sunny, 75°F"

# Use with agents
tools = [multiply_numbers, get_weather]
```

API Integrations

OpenWeatherMap

```
from langchain_community.tools import OpenWeatherMapQueryRun
from langchain_community.utilities import OpenWeatherMapAPIWrapper

weather_tool = OpenWeatherMapQueryRun(
    api_wrapper=OpenWeatherMapAPIWrapper(
        openweathermap_api_key="your-api-key"
    )
)
```

SQL Database

```
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///example.db")
sql_tool = QuerySQLDataBaseTool(db=db)
```

Production Best Practices

1. Error Handling & Retry Logic

```
from langchain.callbacks import get_openai_callback
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=4, max=10)
)
def robust_llm_call(prompt):
    try:
        with get_openai_callback() as cb:
            result = model.invoke(prompt)
            print(f"Tokens used: {cb.total_tokens}")
            return result
    except Exception as e:
        print(f"Error: {e}")
        raise
```

2. Caching for Performance

```
from langchain.cache import SQLiteCache
from langchain.globals import set_llm_cache

# Enable caching
set_llm_cache(SQLiteCache(database_path=".langchain.db"))

# Subsequent identical calls will use cache
result1 = model.invoke("What is AI?")
result2 = model.invoke("What is AI?") # Cached
```

3. Streaming Responses

```
from langchain.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

streaming_model = ChatOpenAI(
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()],
    temperature=0
)

# Responses will stream in real-time
for chunk in streaming_model.stream("Tell me a long story"):
    print(chunk.content, end="", flush=True)
```

4. Monitoring with LangSmith

```
from langsmith import Client

client = Client()

# Trace function execution
@traceable
def my_llm_function(input_text):
    return model.invoke(input_text)

# Run with tracing
result = my_llm_function("Hello, world!")
```

5. Memory Optimization

```
# Use conversation summary for long conversations
from langchain.memory import ConversationSummaryBufferMemory

memory = ConversationSummaryBufferMemory(
    llm=model,
    max_token_limit=2000,
    return_messages=True
)
```

6. Batch Processing

```
# Process multiple requests efficiently
inputs = [{"topic": "AI"}, {"topic": "ML"}, {"topic": "DL"}]
results = chain.batch(inputs)

# Async batch processing
import asyncio

async def async_batch():
    results = await chain.abatch(inputs)
    return results

results = asyncio.run(async_batch())
```

7. Configuration Management

```
# Use environment-specific configs
import os
from dataclasses import dataclass

@dataclass
class LangChainConfig:
    model_name: str = os.getenv("LLM_MODEL", "gpt-3.5-turbo")
    temperature: float = float(os.getenv("LLM_TEMPERATURE", "0.7"))
```

```

max_tokens: int = int(os.getenv("LLM_MAX_TOKENS", "1000"))
vector_store_path: str = os.getenv("VECTOR_STORE_PATH", "./vectorstore")

config = LangChainConfig()

```

8. Security Best Practices

```

# Input validation
def validate_input(user_input: str) -> bool:
    # Check for malicious patterns
    forbidden_patterns = ["DROP TABLE", "DELETE FROM", "<script>"]
    return not any(pattern in user_input.upper() for pattern in forbidden_patterns)

# Sanitize outputs
def sanitize_output(output: str) -> str:
    # Remove sensitive information patterns
    import re
    # Remove potential API keys or tokens
    output = re.sub(r'[A-Za-z0-9]{32,}', '[REDACTED]', output)
    return output

```

9. Testing Strategies

```

import pytest
from unittest.mock import Mock, patch

def test_chain_execution():
    # Mock the LLM response
    with patch.object(model, 'invoke', return_value="Mocked response"):
        result = chain.invoke({"topic": "testing"})
        assert "Mocked response" in str(result)

def test_memory_persistence():
    memory = ConversationBufferMemory()
    memory.save_context({"input": "test"}, {"output": "response"})

    variables = memory.load_memory_variables({})
    assert "test" in variables["history"]

```

10. Deployment Considerations

```

# Use LangServe for API deployment
from langserve import add_routes
from fastapi import FastAPI

app = FastAPI(title="LangChain API")

# Add chain as API endpoint
add_routes(app, chain, path="/chain")

# Health check endpoint

```

```
@app.get("/health")
def health_check():
    return {"status": "healthy"}

# Run with: uvicorn main:app --host 0.0.0.0 --port 8000
```

Quick Reference Commands

Installation Commands

```
# Basic installation
pip install langchain

# With specific providers
pip install langchain-openai langchain-anthropic

# Community integrations
pip install langchain-community

# Experimental features
pip install langchain-experimental

# CLI tools
pip install langchain-cli
```

Common Imports

```
# Core
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# Models
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

# Memory
from langchain.memory import ConversationBufferMemory

# Chains
from langchain.chains import LLMChain, RetrievalQA

# Agents
from langchain.agents import AgentExecutor, create_openai_functions_agent

# Vector stores
from langchain_community.vectorstores import FAISS, Chroma
from langchain_pinecone import PineconeVectorStore

# Document processing
```

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

This handbook provides a comprehensive overview of LangChain for AI/ML developers and automation experts. Each section includes practical code examples you can immediately integrate into your automation workflows with n8n, [Make.com](#), or direct API integrations.